

LangChain Guide

Current practical notes on models, tools, agents, RAG, memory, LangGraph, middleware and production.

Research-based study notes • Updated September 2026

1. What LangChain Is

LangChain is an open-source framework for building LLM applications and agents. Current LangChain documentation emphasizes model interfaces, tools, agents, middleware and integrations. LangChain agents are built on LangGraph's runtime.

Mental model

LLM + prompt/context + tools + state + control flow = application/agent. LangChain provides abstractions so you can compose these pieces without writing every integration from scratch.

2. Installation & Project Setup

```
python -m venv .venv
source .venv/bin/activate      # macOS/Linux
# .venv\Scripts\activate      # Windows
pip install -U langchain
pip install -U langchain-openai
```

Current LangChain Python documentation requires Python 3.10+. Provider integrations are installed as separate packages.

3. Models & Messages

Models are the reasoning/generation component. A common pattern is to initialize a chat model, then invoke it with messages or use it inside an agent.

```
from langchain.chat_models import init_chat_model

model = init_chat_model(
    "openai:gpt-5.4",
    temperature=0.2
)

response = model.invoke("Explain TCP in simple terms")
print(response.content)
```

Model identifiers and provider packages change over time. Prefer the current provider-specific LangChain documentation when selecting a model.

4. Prompts & Structured Output

A good prompt specifies role, task, constraints, input format and desired output. For applications, structured output is often safer than asking the model to emit arbitrary text.

Prompt checklist

- Define the task precisely.
- Separate instructions from untrusted user/data content.
- Specify output schema when downstream code consumes the result.
- Include examples only when they clarify the desired behavior.
- Keep retrieved context relevant and bounded.

5. Tools & Tool Calling

Tools expose functions that an agent can call. The model decides when to call a tool and supplies its arguments according to the tool schema.

```

from langchain.tools import tool

@tool
def search_database(query: str, limit: int = 10) -> str:
    """Search the customer database."""
    return f"Results for {query}, limit={limit}"

```

Useful tools include database queries, web/search APIs, calculators, internal business APIs and code execution—provided they are secured and appropriately scoped.

6. Agents

An agent combines a model and tools in a loop: the model reasons about the next step, invokes tools when needed, observes results, and continues until it reaches a stop condition.

```

from langchain.agents import create_agent

agent = create_agent(
    model="openai:gpt-5.4",
    tools=[search_database],
    system_prompt="You are a helpful data assistant."
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "Find customer records for Akash"}]
})
print(result["messages"][-1].content)

```

When not to use an agent

If the workflow is deterministic—e.g., validate input → call API → transform → save—plain Python or a deterministic workflow can be simpler, cheaper and easier to test.

7. RAG — Retrieval Augmented Generation

RAG retrieves relevant external knowledge and supplies it to the model at inference time. A typical pipeline is: load documents → split text → create embeddings → store vectors → retrieve relevant chunks → generate an answer.

Core components

- Document loader: reads PDF, HTML, files, databases, etc.
- Text splitter: divides content into useful chunks.
- Embedding model: maps text to vectors.
- Vector store: stores and searches embeddings.
- Retriever: returns relevant documents.
- Generator/LLM: answers using retrieved context.

```

# Conceptual RAG flow
documents = load_documents()
chunks = split_documents(documents)
vectors = embed(chunks)
vector_store = store(vectors)

context = vector_store.similarity_search(question)
answer = model.invoke(
    f"Answer using this context:\n{context}\nQuestion: {question}"
)

```

8. PDF RAG Project

LangChain's current knowledge-base tutorial demonstrates semantic search over a PDF and a minimal RAG implementation.

Production improvements

- Preserve document metadata such as page number and source.
- Tune chunk size and overlap using evaluation rather than guesswork.
- Use hybrid search when exact terms matter.
- Rerank retrieved passages when retrieval quality is weak.
- Cite source passages in the final UI.
- Protect against prompt injection in retrieved documents.

9. Memory & State

Conversation history is not the same as durable memory. Short-term conversation state keeps context within an interaction/thread. Long-term memory requires explicit persistence and a design for what information is stored, retrieved and updated.

Current LangChain agents run on LangGraph, which supports persistence/checkpointing and stateful workflows.

10. LangGraph, Middleware & LangSmith

LangGraph is the lower-level orchestration runtime underneath LangChain agents. It is useful when you need explicit nodes, edges, loops, branching, persistence or human-in-the-loop control.

Middleware can add retries, fallbacks, guardrails, logging, rate limits, PII detection and prompt/tool transformations.

LangSmith provides tracing, debugging and evaluation visibility for LLM applications.

11. Evaluation & Production Checklist

- Create a representative test dataset.
- Measure answer correctness, retrieval quality and latency.
- Log tool calls and failures.
- Set timeouts and retry policies carefully.
- Validate tool arguments and permissions.
- Limit tool capabilities by role.
- Protect API keys and secrets.
- Monitor token usage and cost.
- Add human approval for high-impact actions.

Common interview questions

- What is RAG?
- Agent vs chain/workflow?
- What are embeddings?
- What is a vector store?
- Why chunk documents?
- What is tool calling?
- LangChain vs LangGraph?

- How do you reduce hallucinations?
- How do you evaluate an LLM application?

References

- LangChain overview: <https://docs.langchain.com/oss/python/langchain/overview> ■cite■turn1search0■
- LangChain agents: <https://docs.langchain.com/oss/python/langchain/agents> ■cite■turn1search1■
- LangChain semantic search/RAG: <https://docs.langchain.com/oss/python/langchain/knowledge-base> ■cite■turn1search2■
- LangChain quickstart: <https://docs.langchain.com/oss/python/langchain/quickstart> ■cite■turn1search3■
- LangGraph overview: <https://docs.langchain.com/oss/python/langgraph/overview> ■cite■turn1search11■
- LangChain middleware: <https://docs.langchain.com/oss/python/langchain/middleware/overview> ■cite■turn1search8■